

PsycheNavigator

Backend Deep Dive

Complete line-by-line explanation of every backend file
Prepared for code review with AI Senior

Generated: June 23, 2026

Author: Youssef

1. start.py — Server Entry Point

backend/start.py

The very first file Python executes when you launch the backend. It has two critical jobs: (1) load environment variables (the Groq API key) from the .env file so all other modules can access it, and (2) start the uvicorn web server that hosts the FastAPI application. The order matters — `load_dotenv()` **MUST** run before any import that reads the API key.

Line	Code	Explanation
1	<code>import uvicorn</code>	Imports uvicorn, the ASGI web server that runs FastAPI. ASGI (Asynchronous Server Gateway Interface) is the async version of WSGI — it lets the server handle multiple simultaneous requests without blocking. This is essential for our SSE streaming in /chat, where the server holds a connection open while nodes complete one by one. Without uvicorn, FastAPI has no way to listen for HTTP requests.
2	<code>import os</code>	Imports Python's os module for operating system interactions (file paths, environment variables, process management). While not directly used in this file's logic, it's available for any OS-level operations that might be needed.
3	<code>from dotenv import load_dotenv</code>	Imports the load_dotenv function from the python-dotenv library (installed via pip). This function reads a .env file (a plain text file with KEY=VALUE pairs on each line) and injects each pair into Python's environment variables, making them accessible via os.getenv(). This is the industry-standard way to manage secrets — the .env file is in .gitignore so API keys never end up in version control.
4	<code># this is the first file Python executes</code>	Comment documenting that this is the application entry point — the file you run with 'python start.py' to launch the entire backend server.
5	<code># load the environment variables...</code>	Comment explaining that the next line loads the GROQ_API_KEY that llm_factory.py will need when creating LLM instances for each node.
6	<code>load_dotenv()</code>	Reads the .env file in the current directory (backend/) and sets each KEY=VALUE pair as an environment variable. The GROQ_API_KEY gets loaded here. This MUST run before any import that reads the API key — when uvicorn.run() imports api/main.py, that triggers imports of agent/graph.py → all node files → llm_factory.py, which calls os.getenv('GROQ_API_KEY'). If load_dotenv() hasn't run yet, the key would be None and the server would crash on startup with 'No GROQ_API_KEY found'.
7		Empty line for readability.
8	<code>if __name__ == "__main__":</code>	Standard Python guard pattern. When you run 'python start.py', Python sets __name__ to '__main__' so this block executes. But if another file does 'import start', __name__ would be 'start' instead, and this block is skipped — preventing the server from accidentally starting twice. This matters because uvicorn with reload=True re-imports modules internally, and without this guard the server could enter an infinite restart loop.
9	<code>uvicorn.run("api.main:app", ...)</code>	Starts the web server. 'api.main:app' tells uvicorn to import api/main.py and find the 'app' variable (our FastAPI instance). host='0.0.0.0' listens on ALL network interfaces, not just localhost — other devices on your network can reach the server (useful for phone testing). port=8000 means the server is at http://localhost:8000, which the React frontend is configured to call. reload=True watches all .py files and auto-restarts when any changes — this is why you must restart manually only for .env changes (environment variables load once at startup, not on hot-reload). In production you'd set reload=False for performance.

2. llm_factory.py — LLM Configuration

backend/llm_factory.py

A factory function that creates LLM instances. Every node that uses an LLM calls `make_llm()` with a different temperature value. This centralizes the model choice in one place — if you switch from Llama to GPT-4, you change only this file and all 6 LLM-using nodes automatically use the new model. Temperature controls how creative vs deterministic the output is: 0 = always the same answer, 0.7 = varied/creative.

Line	Code	Explanation
1	<code>import os</code>	Imports Python's <code>os</code> module, used here specifically for <code>os.getenv()</code> to read the <code>GROQ_API_KEY</code> environment variable. This key was loaded into the environment by <code>start.py</code> 's <code>load_dotenv()</code> call before any imports happened.
2	<code>from langchain_groq import ChatGroq</code>	Imports <code>ChatGroq</code> from the <code>langchain-groq</code> package — LangChain's official wrapper for Groq's API. This abstraction is key: it translates LangChain's standard interface (<code>.invoke()</code> , <code>prompt llm</code> chains, <code>.with_structured_output()</code>) into Groq REST API calls behind the scenes. If you later switch providers, you'd change this to <code>'from langchain_openai import ChatOpenAI'</code> and update the return line — all node code stays untouched because they only use LangChain's generic interface.
3		Empty line for readability.
4	<code>MODEL = "llama-3.3-70b-versatile"</code>	Module-level constant defining the model ID — Llama 3.3 70B Versatile hosted on Groq's servers. This is the single source of truth for which model the entire app uses. All 6 LLM-using nodes (<code>guard</code> , <code>evaluator</code> , <code>query_builder</code> , <code>psychologist</code> , <code>action_planner</code> , <code>follow_up</code>) use this same model but with different temperatures. To switch models (e.g. <code>'llama-3.1-8b-instant'</code> for faster/cheaper inference during development), change only this line.
5		Empty line for readability.
6		Empty line for readability.
7	<code>def make_llm(temperature: float = 0):</code>	Factory function — creates and returns a configured LLM object. Default temperature=0 (deterministic). Each node calls this at import time with its own temperature: guard=0 (yes/no classification must be consistent), evaluator=0 (quality judgments should be stable across runs), query_builder=0.3 (slight creativity to rephrase differently), psychologist=0.5 (balanced warmth + strict rule-following), action_planner=0.5 (structured but natural), follow_up=0.6 (most creative for conversational variety so follow-up questions don't feel formulaic).
8	<code>"""creates a new LLM instance..."""</code>	Docstring documenting the temperature ranges and which nodes use which values. Good practice for other developers who need to understand the design decision behind each temperature choice.
9	<code>...docstring continued...</code>	Continued docstring listing the temperature=0 (deterministic) and temperature=0.3-0.7 (creative) ranges.
10	<code>...docstring continued...</code>	Continued docstring.
11	<code>api_key = os.getenv("GROQ_API_KEY")</code>	Reads the Groq API key from environment variables. <code>os.getenv()</code> returns <code>None</code> if the key isn't found, unlike <code>os.environ["GROQ_API_KEY"]</code> which would raise a <code>KeyError</code> exception. The key was loaded by <code>start.py</code> 's <code>load_dotenv()</code> . This line runs 6 times at server startup — once per node that calls <code>make_llm()</code> at module level.
12	<code>if not api_key:</code>	Checks if the key is <code>None</code> (not set) or empty string (set but blank). Python's <code>'not'</code> operator treats both <code>None</code> and <code>''</code> as falsy. This defensive check catches the most common setup mistake — forgetting to create a <code>.env</code> file or leaving the key value empty.

Line	Code	Explanation
13	<code>raise RuntimeError("No GROQ_API_KEY...")</code>	Raises a RuntimeError with a clear, actionable error message. This is the fail-fast principle : crash immediately at startup with 'No GROQ_API_KEY found' rather than letting the server start and then failing with a cryptic 'groq.AuthenticationError: Invalid API Key' when the first user sends a message 5 minutes later. The error appears in the terminal the moment you run start.py.
14	<code>return ChatGroq(model=MODEL, ...)</code>	Creates and returns a new ChatGroq instance with the model name, temperature, and API key. Important: each call creates a separate LLM object — the guard's LLM (temp=0) and psychologist's LLM (temp=0.5) are distinct instances. These objects are created once at import time in each node file, NOT per-request — so the server reuses the same 6 LLM objects across all incoming requests, avoiding the overhead of re-creating them.

3. agent/state.py — Shared State Definition

backend/agent/state.py

Defines the shape of the shared memory that flows between all nodes in the graph. This is a TypedDict — it specifies field names and their types, but NOT their values. Think of it as a blueprint. The actual starting values are set in api/main.py for each new request. Each node reads the fields it needs, updates its own fields, and returns the state — LangGraph handles passing it to the next node.

Line	Code	Explanation
1	from typing import TypedDict, List, Optional	Imports Python typing tools. TypedDict creates a dictionary class with specific named keys and enforced types — unlike a regular dict, IDE autocompletion and type checkers know which fields exist. List is the type hint for lists (e.g. List[str] = list of strings). Optional means the value can be the specified type OR None (e.g. Optional[str] = either a string or None).
2		Empty line for readability.
3		Empty line for readability.
4	class AgentState(TypedDict):	Defines the state class that LangGraph uses as shared memory. Every node's run() function receives an AgentState dict as input and returns one as output. Nodes communicate only through this state — they never call each other directly. LangGraph manages the state lifecycle: it initializes it (in api/main.py), passes it to each node, merges the returned updates, and passes the updated state to the next node in the graph.
5	"""shared memory passed between all nodes..."""	Docstring explaining the state's role as the single communication channel between nodes. This is a key architectural decision — no node has a reference to any other node.
6	...docstring continued...	Continued docstring.
7	...docstring continued...	Continued docstring.
8		Empty line for readability.
9	messages: List[dict]	The full conversation history. Each dict has 'role' ('user' or 'assistant') and 'content' (the text). The frontend sends ALL messages on every request (not just the latest) so the psychologist and follow_up nodes can build multi-turn context. Capped to the last 10 messages by api/main.py's MAX_HISTORY to stay within token limits.
10	current_query: str	The search query sent to ChromaDB. Initially set to the user's raw message (e.g. 'My manager keeps dismissing my ideas'). If the evaluator rejects the chunks, the query_builder overwrites this with a rephrased version (e.g. 'expressing needs when feeling dismissed'). The retriever always reads this field, so rephrasing it automatically changes what gets searched.
11	retrieved_chunks: List[dict]	Book chunks returned from ChromaDB after filtering. Each dict has three keys: 'text' (the chunk content), 'source_book' (e.g. 'Nonviolent Communication'), and 'page' (page number). Written by the retriever node, read by the evaluator (to judge quality) and the psychologist (to generate advice from). Replaced entirely on each retrieval attempt — not appended.
12	retrieval_attempts: int	Counter tracking how many times we've searched ChromaDB for this message. Starts at 0, incremented by the retriever after each search. The evaluator checks if this is >= 3 to decide whether to give up and ask the user for more details. Maximum of 3 attempts means the user sees at most 3 different sets of chunks before the system gives up.

Line	Code	Explanation
13	<code>is_enough: bool</code>	The evaluator's binary verdict. True = the chunks are good enough to generate advice → graph routes to psychologist. False = chunks are insufficient → graph routes to query_builder for retry (or END if 3 attempts exhausted). This single boolean controls the main branching point in the pipeline — it determines whether the user gets advice or a retry.
14	<code>retry_reason: Optional[str]</code>	One-sentence explanation from the evaluator about why the chunks were insufficient (e.g. 'excerpts are about dating, not workplace dynamics'). The query_builder reads this to understand what went wrong and craft a better search query. Without this feedback, the query_builder would be rephrasing blindly. Optional because it's None when chunks are sufficient.
15	<code>active_frameworks: List[str]</code>	List of book names that were actually cited in the psychologist's response (e.g. ['Nonviolent Communication', 'Feeling Good']). Detected by checking if each book name appears as a substring in the response text. The frontend displays these as 'Sources:' badges below the advice. Only books from retrieved_chunks are candidates — the psychologist can't cite books it wasn't given.
16	<code>action_plan: Optional[List[str]]</code>	Concrete action steps generated by the action_planner node. Each string is one actionable step like 'Schedule a meeting with your manager to discuss your ideas'. The frontend renders these as a numbered checklist. Optional because it's None until the action_planner runs, and set to [] (empty list) if the LLM fails to generate valid JSON.
17	<code>final_response: Optional[str]</code>	The main text response the user reads. Usually set by the psychologist node (therapy-style advice), but also set by the guard (canned off-topic rejection) or the evaluator (fallback 'I need more context' message after 3 failed retrieval attempts). The frontend displays this as the main chat bubble.
18	<code>should_loop: bool</code>	Multi-turn conversation control flag. Set to False by the follow_up node after each turn completes. Could be used to trigger additional processing rounds, but currently always set to False — each request is a single pass through the pipeline.
19	<code>turn_count: int</code>	Counter for completed conversation turns. Incremented by the follow_up node (the last node before END). Could be used to change behavior after N turns (e.g. suggest professional help after 5 turns), but currently only tracked, not acted upon.
20	<code>is_off_topic: bool</code>	Set by the guard node after classifying the message. If True, the routing function route_after_guard() returns END, skipping the entire pipeline (no retrieval, no LLM advice). If False, the graph continues to the retriever. This is the very first decision point in the pipeline and can short-circuit everything.
21	<code>follow_up_question: Optional[str]</code>	The follow-up question generated by the follow_up node (the final node). Displayed separately from the main advice in the frontend — typically shown below the action plan. Examples: 'What feels like the hardest part of this to try first?' Optional because it's None until the follow_up node runs, and set to None if the evaluator gives up (no follow_up generated).

4. agent/graph.py — Pipeline Orchestration

backend/agent/graph.py

Defines the LangGraph state machine — which nodes exist, how they connect via edges, and the routing functions that decide which path to take at decision points. Edges define which paths are **possible**; routing functions decide which path to **actually take** based on state values. Think of edges as train tracks laid down in advance, and routing functions as the switch operator who flips the track.

Line	Code	Explanation
1	from langgraph.graph import StateGraph, END	Imports the two core LangGraph classes. StateGraph is the graph builder — you register nodes and edges on it, then compile it into a runnable pipeline. END is a special sentinel constant that means 'stop the graph and return the final state'. When a routing function returns END, no more nodes execute and the response is sent back to the API.
2	from agent.state import AgentState	Imports the TypedDict that defines the state shape. StateGraph needs this to know what fields the state contains, so it can properly merge state updates returned by each node.
3	from agent.nodes import ...	Imports all 7 node modules (guard, retriever, evaluator, query_builder, psychologist, action_planner, follow_up). Each module has a run(state) function that the graph will call. By importing them here, all node code (including their LLM initialization) runs at import time when the server starts.
4		Empty line for readability.
5		Empty line for readability.
6	# Routing functions	Comment header for the two routing functions that control the graph's branching logic.
10	def route_after_guard(state) -> str:	First routing function. Called by LangGraph immediately after the guard node completes. Reads the state and returns a string telling the graph which node to run next. LangGraph matches this string against the edge map defined in add_conditional_edges().
11	"""if off-topic → stop..."""	Docstring summarizing the two possible outcomes.
12	return END if state.get("is_off_topic") else "retriever"	One-line conditional: if the guard flagged the message as off-topic → return END (graph stops, user gets the canned rejection). If relevant → return 'retriever' (proceed to search ChromaDB). Uses .get() instead of [] to safely handle the case where is_off_topic hasn't been set yet (returns None, which is falsy, so it would proceed to retriever).
15	def route_after_evaluation(state) -> str:	Second routing function — the most complex decision point. Called after the evaluator judges the chunks. Has three possible outcomes , making this the central branching logic of the entire pipeline.
19	if state["is_enough"]: return "psychologist"	First check: if the evaluator said SUFFICIENT, route to psychologist to generate advice. This is the happy path — chunks are good, proceed to the core response generation.
21	if state["retrieval_attempts"] >= 3: return END	Second check: if we've already tried 3 times and still can't get good chunks, give up. The evaluator has already set final_response to a fallback message ('I need more context...'), so returning END sends that to the user instead of bad advice.
23	return "query_builder"	Default: we have retries left, so route to query_builder to rephrase the search query and try again. This creates the retry loop: evaluator → query_builder → retriever → evaluator.

Line	Code	Explanation
39	<code>def build_graph():</code>	Main function that constructs and returns the compiled graph. Called exactly once in <code>api/main.py</code> when the server starts. The compiled graph object is reused for every incoming request — it's a template that gets executed with fresh <code>initial_state</code> each time.
40	<code>graph = StateGraph(AgentState)</code>	Creates a new empty graph that uses <code>AgentState</code> as its shared memory type. At this point, the graph has no nodes and no edges — just a type definition for the state.
43	<code>graph.add_node("guard", guard.run)</code>	Registers the guard node. Maps the string name 'guard' to the function <code>guard.run</code> . When the graph needs to execute the 'guard' step, it calls <code>guard.run(state)</code> and uses the returned state as the updated state. The same pattern repeats for all 7 nodes below.
44	<code>graph.add_node("retriever", retriever.run)</code>	Registers the retriever node — ChromaDB vector search, no LLM involved.
45	<code>graph.add_node("evaluator", evaluator.run)</code>	Registers the evaluator node — quality gate that judges retrieved chunks.
46	<code>graph.add_node("query_builder", query_builder.run)</code>	Registers the query_builder node — rephrases search queries on retry.
47	<code>graph.add_node("psychologist", psychologist.run)</code>	Registers the psychologist node — the core advice generator.
48	<code>graph.add_node("action_planner", action_planner.run)</code>	Registers the action_planner node — converts advice to structured steps.
49	<code>graph.add_node("follow_up", follow_up.run)</code>	Registers the follow_up node — generates one follow-up question.
54	<code>graph.set_entry_point("guard")</code>	Sets the guard as the first node to run when the graph starts. Every request begins here, regardless of the message content. Without this line, <code>LangGraph</code> wouldn't know where to begin execution.
57	<code>graph.add_conditional_edges("guard", route_after_guard, {...})</code>	Adds a conditional edge from guard. After guard completes, <code>LangGraph</code> calls <code>route_after_guard(state)</code> to get a string, then looks up that string in the map <code>{'retriever': 'retriever', 'END': END}</code> to find the next node. The map's keys are what the routing function can return; the values are the actual node names to execute. This two-step lookup allows decoupling the routing logic from node names.
63	<code>graph.add_edge("retriever", "evaluator")</code>	Fixed edge — after retriever, ALWAYS go to evaluator. No routing function needed because there's only one possible next step. The evaluator must judge every set of chunks, whether it's the first attempt or a retry.
66	<code>graph.add_conditional_edges("evaluator", route_after_evaluation, {...})</code>	The most important conditional edge. Three possible destinations: psychologist (chunks approved), query_builder (retry with rephrased query), or END (3 retries exhausted, send fallback message). This edge, combined with the <code>query_builder</code> → <code>retriever</code> edge below, creates the self-correcting retry loop.
72	<code>graph.add_edge("query_builder", "retriever")</code>	The retry loop edge . After <code>query_builder</code> rephrases the query, go back to retriever to search again with the new query. Combined with the evaluator's conditional edge, this creates the cycle: <code>retriever</code> → <code>evaluator</code> → <code>query_builder</code> → <code>retriever</code> → <code>evaluator</code> → ... (up to 3 times).
75	<code>graph.add_edge("psychologist", "action_planner")</code>	Fixed edge. After the psychologist generates advice, always proceed to create an action plan from that advice.
78	<code>graph.add_edge("action_planner", "follow_up")</code>	Fixed edge. After the action plan is created, always generate a follow-up question.
81	<code>graph.add_edge("follow_up", END)</code>	Final fixed edge. After the follow_up question is generated, the pipeline is complete. <code>LangGraph</code> returns the final state to the API, which streams it to the frontend.
84	<code>return graph.compile()</code>	Compiles the graph definition into a runnable object. Compilation validates all edges (checks for missing nodes, unreachable nodes, etc.) and optimizes the execution plan. The returned object has <code>.astream()</code> and <code>.invoke()</code> methods that <code>api/main.py</code> uses to execute the pipeline.

5. agent/logger.py — Colored Terminal Logging

backend/agent/logger.py

Provides colored, structured logging functions used by every node to print what it's doing to the terminal. This creates the readable output you see during request processing — the boxes (■■■■NODE: GUARD■■■■), colored tags ([GUARD]), checkmarks (✓), and X marks (X). Uses ANSI escape codes that modern terminals interpret as color/style instructions.

Line	Code	Explanation
1	# ANSI escape codes for colored terminal output	Comment explaining the purpose of the constants below. ANSI codes are special character sequences that terminals interpret as formatting instructions rather than printable text.
4	RESET = "\033[0m"	The reset code — turns off all formatting (color, bold, dim) and returns to the terminal's default text style. Every colored log line ends with this so the next line isn't accidentally colored. Without RESET, a green log line would make ALL subsequent terminal output green.
5	BOLD = "\033[1m"	Makes text bold/bright. Used for node tags like [GUARD] and for field labels to make them stand out from their values in the log output.
6	DIM = "\033[2m"	Makes text dimmer/faded. Used for input values to visually distinguish them from labels — the label is bold, the value is dim, creating a clear visual hierarchy.
7	CYAN = "\033[96m"	Cyan (light blue) text color. Used for INFO logs and the node start boxes (■■■■NODE■■■■). Cyan was chosen for general information because it's visible but not alarming.
8	YELLOW = "\033[93m"	Yellow text. Used for INPUT logs — what data a node received. Yellow signals 'attention/data' without implying success or failure.
9	GREEN = "\033[92m"	Green text. Used for SUCCESS logs (✓ checkmark). Green universally means 'good/passed' — when you see green, the node succeeded.
10	RED = "\033[91m"	Red text. Used for FAILURE logs (X X mark). Red means something went wrong — chunks rejected, message off-topic, retries exhausted. Immediately visible in the terminal scroll.
11	BLUE = "\033[94m"	Blue text. Used for ROUTE logs (→ next node). Blue signals 'navigation/flow' — which node the graph is going to next.
12	MAGENTA = "\033[95m"	Magenta/purple text. Used for LLM logs — what the LLM returned. Distinct color separates LLM outputs from everything else.
13	ORANGE = "\033[33m"	Orange text. Defined but not currently used by any logging function — available for future log types.
16	def _tag(node, color):	Private helper function (underscore prefix = internal). Creates a formatted tag like [GUARD] or [RETRIEVER] with color and bold styling. Every public log function calls this to prefix its output with a consistent, colored node identifier.
18	return f"{color}{BOLD}[{node}]{RESET}"	Wraps the node name in: color code → bold → brackets → reset. The RESET at the end ensures only the tag is colored, not the rest of the log line. Example output for _tag('GUARD', GREEN): the text '[GUARD]' appears in bold green.
21	def log_node_start(node):	Prints a visible ASCII box when a node begins execution. This creates the ■■■■■■■■■■ ■ NODE: GUARD ■ ■■■■■■■■■■ output that visually separates each node's logs in the terminal, making it easy to scan through a long request trace.

Line	Code	Explanation
26	<code>width = 58</code>	Fixed width for the box (58 characters). All boxes are the same width regardless of node name length, creating a clean, aligned appearance in the terminal.
29	<code>print(f"█{bar}█", flush=True)</code>	Prints the top border of the box. flush=True forces Python to send this line to the terminal immediately rather than buffering it. Without flush, Python might hold onto the output and you'd see log lines appear out of order or all at once instead of in real-time as nodes execute.
34	<code>def log_input(node, label, value):</code>	Logs what data a node received. Printed in yellow. Example: [GUARD] INPUT user message: 'My manager...'. Called at the start of each node to show what it's working with.
39	<code>def log_llm(node, label, value):</code>	Logs what the LLM returned. Printed in magenta. Example: [GUARD] LLM classification: RELEVANT. Called after every LLM invocation to show the raw LLM output before the node processes it.
44	<code>def log_ok(node, label, value):</code>	Logs a success result with a ✓ checkmark in green. Example: [EVALUATOR] ✓ verdict: SUFFICIENT. Called when a node's decision/output is positive — message is relevant, chunks are good enough, etc.
49	<code>def log_warn(node, label, value):</code>	Logs a failure/problem with a ✗ X mark in red. Example: [EVALUATOR] ✗ verdict: INSUFFICIENT. Called when something didn't pass — off-topic message, bad chunks, retries exhausted.
54	<code>def log_info(node, label, value):</code>	Logs general information in cyan. Example: [RETRIEVER] INFO raw hits from ChromaDB: 8. Used for neutral data that isn't a success or failure — counts, attempt numbers, chunk previews.
59	<code>def log_route(node, destination):</code>	Logs which node runs next, in blue with an arrow. Example: [EVALUATOR] ROUTE → psychologist. Called at the end of nodes that have conditional routing, so you can trace the graph's execution path in the terminal.

6. agent/nodes/guard.py — Off-Topic Filter

backend/agent/nodes/guard.py

The first node in the pipeline. Asks the LLM a simple classification question: 'Is this message about personal or interpersonal psychology?' If yes → continue to retriever. If no → stop the graph immediately with a polite canned rejection. This prevents the system from wasting API tokens and ChromaDB queries on messages like 'What's the weather?' or 'Help me write Python code.'

Line	Code	Explanation
1	<code>from langchain_core.prompts import ChatPromptTemplate</code>	Imports ChatPromptTemplate from LangChain — a class that creates reusable prompt templates with {variable} placeholders. At invocation time, the variables are filled in with actual values and the complete prompt is sent to the LLM. This is cleaner than f-strings because LangChain handles escaping, validation, and chaining with the operator.
2	<code>from agent.state import AgentState</code>	Imports the shared state TypedDict so the run() function can declare its input/output type. This enables IDE autocompletion for state field names inside the function.
3	<code>from llm_factory import make_llm</code>	Imports the factory function to create a configured LLM instance. The guard creates its own LLM with temperature=0 on the next line.
4	<code>from agent.logger import ...</code>	Imports 6 specific logging functions this node uses: log_node_start (box), log_input (yellow), log_llm (magenta), log_ok (green ✓), log_warn (red X), log_route (blue →). Each node only imports the log types it actually calls.
6	<code>llm = make_llm(temperature=0)</code>	Creates the guard's LLM at module level (import time, not per-request). Temperature 0 = fully deterministic — the same message should always get the same classification. This matters for consistency: you don't want 'My manager dismissed my idea' to be classified as 'relevant' on one request and 'off_topic' on the next. The LLM object is created once and reused for all requests.
8	<code>_OFF_TOPIC_MESSAGE = "I'm here to help..."</code>	A hardcoded string , not LLM-generated. When a message is off-topic, this exact text is returned every time. Using a fixed string instead of asking the LLM to generate a rejection ensures: (1) consistent user experience, (2) no wasted API tokens on rejection messages, (3) no risk of the LLM accidentally engaging with the off-topic question.
15	<code>prompt = ChatPromptTemplate.from_template("""...</code>	Opens the classification prompt template. The prompt design is deliberately simple: tell the LLM its role (filter), give clear examples of both categories, and demand exactly one word back. Short prompts use fewer tokens and get faster, more reliable responses for binary classification tasks.
19	<code>RELEVANT: conflicts with a boss, coworker, partner...</code>	Explicit examples of relevant topics. Without examples, the LLM might be too strict (rejecting 'I feel lonely') or too loose (accepting 'Tell me about cognitive psychology as an academic field'). The examples set the boundary: personal/interpersonal situations involving emotions and relationships.
20	<code>IRRELEVANT: sports, weather, news, politics...</code>	Explicit examples of off-topic categories. Including 'coding' is important because users might try to use the chatbot as a general assistant. These examples teach the LLM that we specifically want psychology-related personal situations.
22	<code>User message: {user_message}</code>	The template variable that gets replaced with the actual user message at runtime. ChatPromptTemplate.from_template() automatically detects {user_message} and requires it in the .invoke() call.
24	<code>Return exactly one word: relevant or off_topic</code>	Forces a single-word response — critical for reliable parsing. If the prompt said 'explain your reasoning', the LLM might return paragraphs of text that are harder to parse. 'Exactly one word' makes the response deterministic and easy to check with a simple string comparison.

Line	Code	Explanation
27	<code>def run(state: AgentState) -> AgentState:</code>	The entry point that LangGraph calls. Takes the shared state, performs classification, updates relevant fields, and returns the state. The <code>-> AgentState</code> return type annotation tells LangGraph what to expect back.
31	<code>user_message = state["messages"][-1]["content"]</code>	Extracts the most recent message's text. <code>state['messages']</code> is the full conversation history (list of dicts), <code>[-1]</code> gets the last one (the new message the user just sent), <code>['content']</code> gets the text string. This is the message that needs to be classified.
32	<code>state["current_query"] = user_message</code>	Copies the raw user message into <code>current_query</code> — this is what the retriever will search ChromaDB with. Set here (not in the retriever) because the guard is always the first node and this field needs a value before the retriever runs. If the evaluator later rejects chunks, the <code>query_builder</code> will overwrite this with a rephrased version.
33	<code>state["retrieval_attempts"] = 0</code>	Resets the retry counter to 0 for this new message. Even in multi-turn conversations, each new message gets 3 fresh retrieval attempts.
39	<code>response = (prompt llm).invoke({"user_message": user_message})</code>	LangChain's pipe operator (<code> </code>) creates a chain: prompt llm . When <code>.invoke()</code> is called: (1) prompt fills <code>{user_message}</code> with the actual text, (2) the filled prompt is sent to the LLM, (3) the LLM returns a response object. This is equivalent to <code>llm.invoke(prompt.format(user_message=...))</code> but more composable and cleaner.
40	<code>classification = response.content.strip().lower()</code>	Extracts the LLM's text response (<code>.content</code>), removes whitespace (<code>.strip()</code>), and lowercases (<code>.lower()</code>). Normalization ensures we can reliably check for 'off_topic' regardless of whether the LLM returns 'Off_Topic', 'OFF_TOPIC', or 'off_topic'.
44	<code>if "off_topic" in classification:</code>	Uses 'in' instead of '==' as a safety measure. If the LLM returns 'off_topic.' with a period, or 'off_topic (the message is about sports)', the 'in' check still catches it. A strict <code>==</code> comparison would miss these edge cases and let off-topic messages through to the retriever.
46	<code>state["is_off_topic"] = True</code>	Sets the flag that <code>route_after_guard()</code> reads. When this is True, the routing function returns END and the graph stops — no retrieval, no psychologist, no follow-up. The user just gets the canned <code>_OFF_TOPIC_MESSAGE</code> .
47	<code>state["final_response"] = _OFF_TOPIC_MESSAGE</code>	Stores the canned rejection as the response. Since the graph stops at END, no other node will overwrite this. The API returns this text to the frontend as the assistant's message.
48	<code>state["action_plan"] = []</code>	Sets empty action plan — there's no advice to create action steps for when the message is off-topic. The frontend checks this field and shows nothing if the list is empty.
49	<code>state["active_frameworks"] = []</code>	Sets empty frameworks — no books were consulted. The frontend's 'Sources:' section won't display anything.
53	<code>state["is_off_topic"] = False</code>	Marks the message as relevant. <code>route_after_guard()</code> will read this and return 'retriever', continuing the pipeline to search for book excerpts.
57	<code>return state</code>	Returns the updated state to LangGraph. The graph engine then calls <code>route_after_guard(state)</code> to decide the next node: retriever (relevant) or END (off-topic).

7. agent/nodes/retriever.py — ChromaDB Vector Search

backend/agent/nodes/retriever.py

Searches ChromaDB for the 8 most similar book chunks to the current query. Then applies two filters: (1) reject chunks from unknown book sources, (2) reject chunks that mention foreign book titles as cross-references. This is the ONLY node that does NOT use an LLM — it's pure vector similarity search. The embedding model converts the query into a 384-dimensional vector and ChromaDB finds the closest matches.

Line	Code	Explanation
1	<code>from rag.retriever import build_retriever</code>	Imports the getter function that returns the pre-initialized ChromaDB retriever from rag/retriever.py. That module loads the embedding model and connects to ChromaDB at import time, so by the time this line runs, everything is ready.
2	<code>from agent.state import AgentState</code>	Imports the shared state type.
3	<code>from agent.logger import ...</code>	Imports logging functions: log_node_start, log_input, log_info, log_ok. Note this node doesn't import log_warn or log_llm — it never fails (ChromaDB always returns results) and doesn't use an LLM.
5	<code>_retriever = build_retriever()</code>	Gets the retriever object at module level (import time). This is critical for performance — the embedding model (all-MiniLM-L6-v2, ~80MB) takes several seconds to load. By loading once at startup and reusing for every request, we avoid re-loading the model on each query. If this were inside run(), every search would take seconds longer.
7	<code>_KNOWN_BOOKS = {...}</code>	A Python set containing the 6 valid book names: Feeling Good, Attached, The Body Keeps the Score, Games People Play, Thinking Fast and Slow, Nonviolent Communication. Using a set (not a list) makes 'in' lookups O(1) instead of O(n). Any chunk whose metadata 'source' field doesn't match one of these names is rejected — this protects against corrupted or test data in ChromaDB.
17	<code>_FOREIGN_MARKERS = [...]</code>	List of book titles that appear as cross-references inside our PDF books (e.g. 'Radical Acceptance', 'Crucial Conversations', 'The Gifts of Imperfection'). When an NVC excerpt mentions 'as described in Crucial Conversations...', that chunk's content is about a foreign book, not about NVC itself. Filtering these out ensures the psychologist only sees content actually written by our 6 books' authors.
33	<code>def _has_foreign_reference(text):</code>	Helper function that loops through _FOREIGN_MARKERS and returns True if any foreign title appears as a substring in the chunk text. Simple 'marker in text' check — fast and effective for exact title matches.
42	<code>def _filter_chunks(results):</code>	The main filter function. Takes raw ChromaDB Document objects and applies two filters : (1) reject if source metadata isn't in _KNOWN_BOOKS, (2) reject if text mentions a foreign book title. Returns a tuple: (list of clean chunk dicts, count of filtered-out chunks). Each clean chunk is a dict with 'text', 'source_book', and 'page' keys — a simpler format than LangChain's Document objects.
66	<code>def run(state: AgentState) -> AgentState:</code>	Entry point called by LangGraph.
68	<code>attempt_num = state["retrieval_attempts"] + 1</code>	Calculates the human-readable attempt number (1, 2, or 3) for logging. The state field starts at 0 and is incremented at the end of this function, so we add 1 here for display purposes.
72	<code>results = _retriever.invoke(state["current_query"])</code>	The core search operation. Sends the current query to ChromaDB, which: (1) converts the query text into a 384-dimensional vector using the all-MiniLM-L6-v2 model, (2) performs approximate nearest-neighbor search using the HNSW algorithm, (3) finds the 8 vectors most similar by cosine distance, (4) returns their corresponding Document objects with .page_content and .metadata. This entire operation typically takes 20-50ms.

Line	Code	Explanation
75	<code>chunks, filtered_out = _filter_chunks(results)</code>	Runs both filters on the raw results. Typically 0-2 chunks are filtered out. The function returns the clean chunks as simple dicts (easier to serialize to JSON for the frontend) and the count of rejected chunks (for logging).
83	<code>state["retrieved_chunks"] = chunks</code>	Stores the filtered chunks in state. The evaluator will read these to judge quality, and if approved, the psychologist will use them to generate advice. This replaces the previous value entirely — on retry attempts, the old chunks are discarded and replaced with new ones from the rephrased query.
84	<code>state["retrieval_attempts"] += 1</code>	Increments the attempt counter. After this, the evaluator can check: if <code>retrieval_attempts >= 3</code> and chunks are still insufficient, give up. The counter persists across the retry loop (retriever → evaluator → query_builder → retriever) because it's in the shared state.
85	<code>return state</code>	Returns the updated state. The graph always routes to the evaluator next (fixed edge in graph.py).

8. agent/nodes/evaluator.py — Chunk Quality Gate

backend/agent/nodes/evaluator.py

The quality gatekeeper of the pipeline. Asks the LLM: 'Are these book excerpts good enough to give advice for this person's situation?' If SUFFICIENT → continue to psychologist. If INSUFFICIENT → retry with a rephrased query (up to 3 times). After 3 failures → ask the user for more details. The key innovation in the prompt: it accepts **transferable psychological principles**, not just exact scenario matches.

Line	Code	Explanation
1	<code>from langchain_core.prompts import ChatPromptTemplate</code>	Imports the prompt template class. The evaluator's prompt is the longest in the system — it needs careful template variable handling for {user_message} and {chunks}.
2	<code>from agent.state import AgentState</code>	Imports shared state type.
3	<code>from llm_factory import make_llm</code>	Imports LLM factory.
4	<code>from agent.logger import ...</code>	Imports 6 logging functions including both log_ok (for SUFFICIENT) and log_warn (for INSUFFICIENT) since this node can produce either outcome.
6	<code>llm = make_llm(temperature=0)</code>	Temperature 0 for deterministic evaluation. The same chunks should ideally get the same verdict every time — though in practice, LLM outputs at temperature=0 can still vary slightly between runs. This is why the same query sometimes passed and sometimes failed in earlier testing.
8	<code>prompt = ChatPromptTemplate.from_template("""...</code>	Opens the evaluation prompt — the most carefully designed prompt in the system. Every instruction was tuned based on real failure cases.
9	<code>Evaluate whether these book excerpts are useful enough...</code>	The opening instruction. Note it says 'useful enough' not 'perfectly relevant' — this deliberately lower bar was chosen because the books are general psychology, not situation-specific guides. The original prompt said 'strictly evaluate' which caused the evaluator to reject good chunks that had transferable principles.
12	<code>User situation: {user_message}</code>	Template variable for the user's message. The evaluator needs both the user's situation AND the chunks to judge relevance — it's asking 'are THESE chunks good for THIS situation?'
15	<code>Retrieved excerpts: {chunks}</code>	Template variable for all the chunks joined as text, each prefixed with its book name. The evaluator reads all 8 chunks and makes a holistic judgment.
17	<code>The excerpts come from general psychology books...</code>	Critical context paragraph. Without this, the evaluator expected chunks to literally mention 'manager in a meeting' when the user asked about their manager. This line tells the evaluator that the books discuss principles, not specific scenarios — an NVC excerpt about a school meeting is valid for a workplace meeting because the communication principles transfer.
23	<code>Do at least 2 excerpts contain a psychological concept...</code>	The threshold rule : at least 2 applicable chunks. One chunk isn't enough to ground a multi-paragraph response. Requiring all 8 would be impossibly strict. Two is the sweet spot — enough for a substantive, well-supported response.
29	<code>Applicable frameworks from these books include:</code>	Lists the 6 named frameworks (NVC 4 steps, CBT tools, attachment styles, etc.) as a concrete checklist. Without this list, the evaluator might not recognize that a chunk about 'cognitive distortions' from Feeling Good is actually a named CBT framework — it would just see vague psychology text.

Line	Code	Explanation
37	<code>...transferable principles count.</code>	The key loosening rule. An NVC chunk about expressing needs in a school setting counts for a workplace query because the Observation→Feeling→Need→Request framework is universal. Before this rule was added, the evaluator rejected chunks whose surface scenario didn't match, even when the underlying principle was perfectly applicable.
41	<code>Answer with exactly one word on the first line: SUFFICIENT or INSUFFICIENT</code>	Forces a parseable format. Line 1 = verdict (one word), Line 2 = reason (one sentence). The code on line 65-67 splits by newline and reads these two values. If the prompt didn't enforce this format, the LLM might return paragraphs of analysis that are impossible to parse reliably.
48	<code>def run(state: AgentState) -> AgentState:</code>	Entry point.
53	<code>chunks_text = "\n\n".join(...)</code>	Joins all retrieved chunks into one text block, each prefixed with its book name: '[Nonviolent Communication] chunk text here...'. This is what the LLM sees as the 'Retrieved excerpts' in the prompt. Double newlines separate chunks for readability.
58	<code>response = (prompt llm).invoke({...})</code>	Fills the template with user_message and chunks_text, sends to the LLM. The response contains the SUFFICIENT/INSUFFICIENT verdict and the reason.
65	<code>lines = response.content.strip().split("\n")</code>	Parses the LLM's response by splitting on newlines. Expected format: line[0] = 'SUFFICIENT' or 'INSUFFICIENT', line[1] = reason sentence. strip() removes any leading/trailing whitespace that could cause parsing issues.
66	<code>first_line = lines[0].strip().upper()</code>	Normalizes the verdict to uppercase for reliable comparison. Handles cases where the LLM returns 'Sufficient' or 'sufficient' instead of 'SUFFICIENT'.
67	<code>reason = lines[1].strip() if len(lines) > 1 else ""</code>	Safely extracts the reason from line 2. The 'if len(lines) > 1' guard handles the edge case where the LLM returns only the verdict with no explanation — without this check, lines[1] would throw an IndexError.
69	<code>state["is_enough"] = first_line == "SUFFICIENT"</code>	Converts the string verdict to a boolean. True only if the LLM said exactly 'SUFFICIENT'. Any other response (including 'PARTIALLY SUFFICIENT' or 'MAYBE') is treated as False. This boolean is what route_after_evaluation() reads to decide the next node.
70	<code>state["retry_reason"] = reason</code>	Stores the evaluator's explanation. If retrying, the query_builder reads this to understand what was wrong with the chunks (e.g. 'chunks are about dating, not workplace dynamics') and craft a better search query. This feedback loop is what makes the retry mechanism self-correcting.
72	<code>if state["is_enough"]:</code>	SUFFICIENT path — chunks are good.
75	<code>log_route("EVALUATOR", "psychologist")</code>	Logs that the graph will proceed to the psychologist to generate advice.
80	<code>if attempts >= 3:</code>	Checks if all 3 retry attempts have been exhausted. If so, there's no point trying again — the books probably don't have content relevant to this specific query.
83	<code>state["final_response"] = "I want to give you the best advice..."</code>	Sets a graceful fallback response instead of giving advice based on bad chunks. The message asks the user for more details: who's involved, what happened, how they feel. This is better than giving irrelevant advice — it maintains trust and invites the user to rephrase.
91	<code>state["action_plan"] = []</code>	Clears the action plan since no advice was generated. The frontend shows no action items.
92	<code>state["active_frameworks"] = []</code>	Clears the frameworks since no books were used.
93	<code>state["follow_up_question"] = None</code>	Clears the follow-up question. Since the graph routes to END (not through follow_up node), this field would remain None anyway, but setting it explicitly is defensive coding.
97	<code>log_info("EVALUATOR", "attempts used", ...)</code>	Logs the current attempt count and that a retry will happen.

Line	Code	Explanation
98	log_route("EVALUATOR", "query_builder")	Logs that the graph routes to query_builder to rephrase the query.
100	return state	Returns the updated state. route_after_evaluation() uses is_enough and retrieval_attempts to decide the next node.

9. agent/nodes/query_builder.py — Query Rephrasing

backend/agent/nodes/query_builder.py

Called when the evaluator rejects chunks. Takes the failed query, the reason it failed, and the original user message, then asks the LLM to write a better search query. The new query overwrites `current_query` in the state and the retriever searches again. This creates the self-correcting retry loop: evaluator rejects → `query_builder` rephrases → retriever searches → evaluator judges again.

Line	Code	Explanation
1	<code>from langchain_core.prompts import ChatPromptTemplate</code>	Imports the prompt template class for creating the rephrasing prompt with {variable} placeholders.
2	<code>from agent.state import AgentState</code>	Imports shared state type.
3	<code>from llm_factory import make_llm</code>	Imports LLM factory.
4	<code>from agent.logger import ...</code>	Imports logging functions. Uses <code>log_warn</code> (red) for the retry trigger since a retry means the previous attempt failed.
6	<code>llm = make_llm(temperature=0.3)</code>	Temperature 0.3 — needs some creativity to rephrase the query differently from the failed version, but not too much creativity or it generates overly verbose queries stuffed with buzzwords. Higher temperature (0.7+) was tested and produced queries like 'Overcoming psychological reactance in workplace communication including Radical Candor and Crucial Conversations' — long queries that actually performed worse with the embedding model.
8	<code>prompt = ChatPromptTemplate.from_template("""...</code>	Opens the rephrasing prompt. Designed to give the LLM all the context it needs to write a better query.
9	<code>You tried to help with: {user_message}</code>	The original user situation. Anchors the rephrasing — without this, the LLM might drift away from what the user actually asked about. The <code>user_message</code> is always the raw message, never the rephrased query.
10	<code>You searched for: {previous_query}</code>	The query that just failed. The LLM needs to know what was already tried so it doesn't generate the same query again. On the second retry, this would be the first rephrased query.
11	<code>The chunks were insufficient because: {retry_reason}</code>	The evaluator's one-sentence explanation of what was wrong (e.g. 'chunks are about dating, not workplace dynamics'). This is the feedback signal — it tells the LLM specifically what to fix. Without this, the <code>query_builder</code> would be rephrasing blindly.
13	<code>Write a better, more specific search query...</code>	The core instruction. 'More specific' guides toward targeted psychological concepts rather than broad queries.
14	<code>Think: specific psychological concepts, named frameworks...</code>	Nudges the LLM toward searchable terms that exist in the book database — concepts like 'expressing needs', 'cognitive distortions', 'attachment anxiety'. These terms appear in the actual book text and are more likely to produce good embedding matches.
17	<code>Return only the search query, nothing else.</code>	Critical constraint — prevents the LLM from adding explanations like 'Here's a better query: ...' which would pollute the search. The raw output goes directly into <code>state['current_query']</code> and then into ChromaDB.
22	<code>def run(state: AgentState) -> AgentState:</code>	Entry point called by LangGraph after the evaluator says INSUFFICIENT and <code>retrieval_attempts</code> is less than 3.
24	<code>log_warn("QUERY_BUILDER", "retry triggered", ...)</code>	Red X log showing which attempt just failed (e.g. 'attempt 1/3 failed'). The warn color signals that something went wrong that triggered this node.
28	<code>new_query = (prompt llm).invoke({...})</code>	Fills the template with all three context variables and sends to the LLM. The LLM returns a new search query string. The <code>prompt llm</code> chain handles the full pipeline: template filling → LLM call → response extraction.

Line	Code	Explanation
33	<code>state["current_query"] = new_query.content.strip()</code>	Overwrites the current search query with the rephrased version. <code>.strip()</code> removes whitespace. The retriever always reads <code>state['current_query']</code> , so changing this field is all that's needed to make the next search use the new query. The old query is lost — it's not stored anywhere.
36	<code>log_route("QUERY_BUILDER", "retriever (retry)")</code>	Logs that the graph returns to the retriever for another search attempt with the new query.
37	<code>return state</code>	Returns the state. The fixed edge in <code>graph.py</code> sends it directly to the retriever, which will invoke ChromaDB with the new <code>current_query</code> .

10. agent/nodes/psychologist.py — Core Advice Generator

backend/agent/nodes/psychologist.py

The heart of the system — generates the actual therapy-style advice the user reads. Takes the approved book excerpts and full conversation history, then produces a response governed by 4 strict rules: GROUNDING (only use excerpt content), DEMONSTRATION (walk through frameworks step-by-step), SITUATION (use correct relationships), CITATION (proper book references). This is the most complex prompt in the system and the node that directly determines response quality.

Line	Code	Explanation
1	<code>from langchain_core.messages import SystemMessage, HumanMessage, AIMessage</code>	Imports LangChain message types for manually building multi-turn conversations. Unlike other nodes that use ChatPromptTemplate, the psychologist needs to include the full conversation history (past user messages + past assistant responses), which requires constructing a list of typed message objects that the LLM interprets as a back-and-forth dialogue.
2	<code>from agent.state import AgentState</code>	Imports shared state type.
3	<code>from llm_factory import make_llm</code>	Imports LLM factory.
5	<code>llm = make_llm(temperature=0.5)</code>	Temperature 0.5 — a carefully chosen balance. Lower (0.3) made responses feel robotic and formulaic. Higher (0.7) caused the LLM to ignore the strict GROUNDING and CITATION rules, pulling in knowledge from its training data instead of sticking to the excerpts. 0.5 produces warm, natural language while still respecting the 4 rules.
7	<code>_SYSTEM = """You are a compassionate...</code>	Opens the system prompt — a multi-line string containing all instructions the LLM must follow. This is the longest prompt in the system (~60 lines). The {context} variable at the end gets replaced with the actual book excerpts at runtime.
11	<code>Tone: warm, direct, non-judgmental, empowering.</code>	Tone guidelines. 'Direct' prevents the LLM from being wishy-washy. 'Non-judgmental' is essential for a psychology context — users sharing personal struggles need to feel safe, not criticized.
12	<code>Never diagnose. Never prescribe. Always empower.</code>	Safety guardrails. The system is NOT a therapist — it must not say 'you have anxiety disorder' (diagnose) or 'take this medication' (prescribe). 'Empower' means giving the user tools and frameworks they can apply themselves, not making them dependent on the chatbot.
13	<code>Always acknowledge feelings before giving advice.</code>	Therapeutic best practice — before jumping to solutions, validate the person's emotional experience. 'I can sense your frustration' before 'Here's what NVC suggests...' This makes the user feel heard and more receptive to the advice.
15	<code>GROUNDING RULE — CRITICAL:</code>	Rule 1 — the core RAG constraint. Without this rule, the LLM would use its vast training data to generate generic psychology advice, completely ignoring the book excerpts. The grounding rule forces it to derive ALL advice from the provided excerpts, making the response verifiable and source-backed.
16	<code>Your advice MUST come from the book excerpts...</code>	The strongest constraint in the system. 'MUST' and 'Do NOT draw on your own knowledge' are deliberately forceful because LLMs have a strong tendency to supplement with their training data, especially when the excerpts feel incomplete.
22	<code>Skip excerpts that genuinely do not apply...</code>	Permission to ignore irrelevant chunks. Without this line, the LLM would try to force all 8 chunks into the response, leading to stretches like citing Thinking Fast and Slow's 'eliminating redundancy' principle as advice for workplace conflicts. Using fewer excerpts well produces better responses than shoehorning all of them.
25	<code>DEMONSTRATION RULE — CRITICAL:</code>	Rule 2 — show, don't tell. This rule was added after testing showed the LLM consistently name-dropped techniques ('According to NVC, separate observation from evaluation') without actually demonstrating them. The good/bad examples teach by contrast.

Line	Code	Explanation
29	Good: 'Using NVC here, you could say to your manager...'	The good example fills in all 4 NVC steps with the user's specific details: observation (proposals get brief responses), feeling (discouraged), need (contributions considered), request (5 minutes after meeting). This is what a real therapist would do — not explain the framework abstractly but show the user exactly what to say.
33	Bad: 'According to NVC, you should separate observation from evaluation.'	The bad example just names the technique without applying it. The user learns nothing actionable — they still don't know what to actually say to their manager. This pattern was the most common failure mode before this rule was added.
36	Good: 'The distorted thought might be...'	Concrete before/after for CBT cognitive distortions: distorted thought ('My manager thinks I'm incompetent') → realistic reframe ('My manager may be rushed, doesn't mean my ideas lack merit'). Specific to the user's situation, not generic.
39	Bad: 'Try to identify any distorted thought patterns.'	Generic instruction that puts the work on the user instead of helping them. A real therapist would identify the distortion FOR them and show the reframe.
40	Lead with guidance, not questions.	Stops the LLM from volleying questions back at the user ('What do you think the underlying reasons are?'). The user came for advice, not a Socratic dialogue. The psychologist should offer its read of the situation.
43	SITUATION RULE – CRITICAL:	Rule 3 — relationship accuracy. LLMs have a common failure mode where they default to 'your partner' for any interpersonal issue, even when the user said 'my manager'. This rule explicitly forbids relationship swaps.
48	If the user says 'manager', always say 'your manager'. NEVER say 'partner'.	The most explicit instruction in the system. Added after testing revealed the LLM occasionally saying 'discuss this with your partner' when the user was asking about their boss. This error would be confusing and undermine trust.
50	CITATION RULE – CRITICAL:	Rule 4 — proper attribution. Ensures traceability: the user (and reviewers) can verify that advice actually comes from the cited book and page.
52	...ALWAYS cite it as: Book Name (p.X).	Standardized citation format that the frontend can parse and display consistently. The page number adds verifiability — someone could look up page 42 of Nonviolent Communication to check the source.
54	Only cite books from the label at the top of each excerpt...	Prevents a subtle error: if an NVC excerpt mentions 'as described in Crucial Conversations', the LLM should NOT cite Crucial Conversations — that book isn't in our database. Only the label [Nonviolent Communication, p.42] counts as a valid source.
61	{context}"""	The {context} placeholder where all book excerpts get injected at runtime. Each excerpt is formatted as '[Book Name, p.X]\nchunk text'. The closing triple-quote ends the system prompt string.
64	def run(state: AgentState) -> AgentState:	Entry point. This function builds the full conversation, sends it to the LLM, and extracts the response.
65	context = "\n\n".join(...)	Joins all retrieved chunks into a single text block with labels. Each chunk becomes: [Nonviolent Communication, p.42] followed by the actual text. Double newlines between chunks for readability.
70	msgs = [SystemMessage(content=_SYSTEM.format(context=context))]	Creates the conversation's first message — the system prompt with all 4 rules plus the book excerpts injected via Python's .format(). Known risk: .format() will crash if any excerpt contains literal { or } characters. A safer alternative would be to use .replace('{context}', context) or LangChain's ChatPromptTemplate.
71	for msg in state["messages"][:-1]:	Loops through conversation history except the last message (which is added separately on line 78). Converts each dict into the appropriate LangChain message type so the LLM understands the dialogue structure.
74	if role == "user":	User messages become HumanMessage objects — the LLM sees these as things the human said.
76	elif role in ("assistant", "ai"):	Assistant messages become AIMessage objects — the LLM sees these as its own previous responses. Checks for both 'assistant' and 'ai' because different parts of the system may use different role labels.

Line	Code	Explanation
78	<code>msgs.append(HumanMessage(content=state["messages"][-1]["content"]))</code>	Adds the current (latest) user message as the final HumanMessage. This is what the LLM will respond to. It's added last so it's the most recent thing in the conversation — the LLM generates a response to the last HumanMessage.
80	<code>response = llm.invoke(msgs)</code>	Sends the full conversation (system prompt + history + current message) to the LLM via Groq's API. The LLM generates the therapy-style advice following all 4 rules. This is the most token-intensive call in the pipeline — the system prompt alone is ~800 tokens plus all the excerpts.
81	<code>state["final_response"] = response.content</code>	Stores the LLM's text output as the final response. This is the main advice the user reads in the chat interface. If the psychologist runs, this overwrites any previous value (like the guard's off-topic message, though that path would never reach here).
84	<code>all_books = list(set([c["source_book"] for c in state["retrieved_chunks"]]))</code>	Extracts unique book names from all retrieved chunks. The list comprehension gets every source_book, set() removes duplicates, list() converts back. For example: ['Nonviolent Communication', 'Feeling Good', 'Attached'].
85	<code>state["active_frameworks"] = [b for b in all_books if b in response.content]</code>	Filters to only books that appear in the actual response text . If the psychologist skipped the Thinking Fast and Slow chunks (per the GROUNDING rule), that book won't appear here. Known limitation: substring matching means 'Attached' would falsely match inside words like 'unattached' or 'reattached'.
86	<code>return state</code>	Returns the state. The graph routes to action_planner next (fixed edge).

11. agent/nodes/action_planner.py — Structured Action Plan

backend/agent/nodes/action_planner.py

Takes the psychologist's advice and converts it into a structured, step-by-step action plan. Uses LangChain's **structured output** feature to force the LLM to return valid JSON matching a Pydantic schema — not free-form text. This ensures the frontend always receives a clean list of steps it can render as a numbered checklist. Falls back to an empty plan if JSON parsing fails.

Line	Code	Explanation
1	<code>from langchain_core.prompts import ChatPromptTemplate</code>	Imports prompt template class.
2	<code>from pydantic import BaseModel</code>	Imports Pydantic's BaseModel — the foundation for defining data schemas in Python. Pydantic validates that data matches a specific structure (field names, types, constraints). Here it defines the exact JSON shape the LLM must return.
3	<code>from typing import List</code>	Imports List type hint for the steps field (<code>List[str]</code> = a list of strings).
4	<code>from agent.state import AgentState</code>	Imports shared state type.
5	<code>from llm_factory import make_llm</code>	Imports LLM factory.
7	<code>llm = make_llm(temperature=0.5)</code>	Temperature 0.5 — needs to be creative enough to generate specific, varied action steps, but structured enough to produce valid JSON. Higher temperatures risk malformed JSON output.
10	<code>class ActionPlan(BaseModel):</code>	Defines the Pydantic schema that the LLM's response must match. LangChain sends this schema to the LLM as part of the prompt, instructing it to return JSON with exactly these fields.
11	<code>steps: List[str]</code>	A list of action step strings, e.g. ['Schedule a meeting with your manager', 'Practice the NVC request beforehand']. This is the main output — what the frontend displays as the action plan checklist.
12	<code>framework_used: str</code>	Which psychological framework the plan is based on, e.g. 'NVC 4 steps'. Not currently displayed in the frontend but useful for debugging and potential future features.
13	<code>book_source: str</code>	Which book the framework came from, e.g. 'Nonviolent Communication'. Same as above — stored but not currently surfaced.
14	<code>time_horizon: str</code>	When to execute the plan, e.g. 'This week' or 'Before your next meeting'. Helps the user understand urgency.
18	<code>structured_llm = llm.with_structured_output(ActionPlan)</code>	Wraps the LLM with structured output enforcement. Behind the scenes, LangChain adds the ActionPlan JSON schema to the prompt and uses Groq's function-calling feature (or JSON mode) to ensure the response is valid JSON matching the schema. The returned object is an ActionPlan instance, not a string — you access fields like <code>result.steps</code> , <code>result.framework_used</code> , etc.
20	<code>prompt = ChatPromptTemplate.from_template("""...</code>	Opens the prompt template with three variables: {response} (the psychologist's advice), {context} (truncated chunk text), and {user_message} (the original query).
25	<code>Steps must be specific, behavioural, and immediately actionable.</code>	The key constraint. 'Specific' = not vague ('think about your feelings'). 'Behavioural' = observable actions ('schedule a meeting'). 'Immediately actionable' = doable this week, not someday.
26	<code>Not abstract. The person should know exactly what to do tomorrow.</code>	Reinforces the specificity requirement. 'What to do tomorrow' is a concrete test: if a step passes the 'could I literally do this tomorrow morning?' test, it's specific enough.

Line	Code	Explanation
30	<code>def run(state: AgentState) -> AgentState:</code>	Entry point.
31	<code>context = "\n".join([...chunk["text"][:200]...])</code>	Builds context from the first 200 characters of each chunk. The action planner doesn't need full excerpts — just enough to know which frameworks were discussed. Truncating saves tokens and keeps the prompt focused on planning, not re-reading excerpts.
35	<code>try:</code>	Wraps the LLM call in try/except because structured output can fail. If the LLM returns malformed JSON that doesn't match the ActionPlan schema, Pydantic raises a ValidationError. Rather than crashing the entire request, we catch it and return an empty plan.
40	<code>result = (prompt structured_llm).invoke({...})</code>	Sends the prompt to the structured LLM. Returns an ActionPlan object (not a string). The pipeline fills the template, sends to the LLM with the JSON schema, and parses the response into a Pydantic model — all in one chained call.
45	<code>state["action_plan"] = result.steps</code>	Extracts just the steps list from the ActionPlan object and stores it in state. The frontend receives this list and renders each step as a numbered item in the action plan section.
46	<code>except Exception:</code>	Catches any exception — JSON parse errors, Pydantic validation errors, API timeouts, etc. This is a broad catch because we'd rather show no action plan than crash the entire response.
48	<code>state["action_plan"] = []</code>	Falls back to an empty list. The frontend checks if this list is empty and simply doesn't render the action plan section — the user still gets the psychologist's advice and follow-up question, just no structured steps.
49	<code>return state</code>	Returns state. Graph routes to follow_up (fixed edge).

12. agent/nodes/follow_up.py — Follow-up Question

backend/agent/nodes/follow_up.py

The final node before END. Generates ONE follow-up question to keep the conversation going. Reads the full conversation history to avoid asking something already discussed. Uses the highest temperature (0.6) of any node because follow-up questions need conversational variety — the same advice shouldn't always end with the same question.

Line	Code	Explanation
1	<code>from langchain_core.messages import SystemMessage, HumanMessage, AIMessage</code>	Imports message types for building multi-turn conversation, same as the psychologist node. Needed because the follow_up also reads conversation history to avoid repeating previous questions.
2	<code>from agent.state import AgentState</code>	Imports shared state type.
3	<code>from llm_factory import make_llm</code>	Imports LLM factory.
5	<code>llm = make_llm(temperature=0.6)</code>	The highest temperature of any node. Follow-up questions need variety: if every conversation ends with 'How do you feel about trying this?', it feels robotic. 0.6 provides enough randomness that the same advice generates different follow-up angles across sessions.
7	<code>_SYSTEM = """You are a compassionate psychology guide...</code>	Opens the system prompt. Unlike the psychologist's complex 4-rule prompt, this one is simple: you have the advice, you have the plan, ask one good question.
10	<code>Advice: {response}</code>	Template variable injecting the psychologist's full advice text. The follow_up needs to know what advice was given so it can ask a relevant question, not repeat something already covered.
11	<code>Action plan: {action_plan}</code>	Template variable injecting the action steps (joined as text). The follow_up might ask 'Which of these steps feels most challenging?' — it needs to know the steps to reference them.
13	<code>Conversation history is provided below. Read it carefully...</code>	Critical instruction to prevent repetition. In multi-turn conversations, earlier turns might have already asked 'How does that make you feel?' — the follow_up must check and ask something new.
16	<code>Ask ONE short follow-up question that either:</code>	The ONE constraint is important — without it, the LLM might ask 3 questions in one response, which is overwhelming. One focused question encourages the user to continue without feeling interrogated.
17	<code>- Digs deeper into something not yet covered</code>	Option 1: explore a new angle. If the advice covered NVC communication strategies, the follow-up might ask about the user's emotional response.
18	<code>- Checks how they feel about the action plan</code>	Option 2: emotional check-in. 'Does the idea of scheduling that conversation with your manager feel doable?'
19	<code>- Uncovers something that would change your advice</code>	Option 3: information gathering. 'Is this a pattern with your manager, or a recent change?' — the answer might lead to different frameworks on the next turn.
24	<code>def run(state: AgentState) -> AgentState:</code>	Entry point.
25	<code>system_content = _SYSTEM.format(...)</code>	Fills the template with the psychologist's advice and the action plan steps (joined with newlines). If action_plan is None, 'or []' provides an empty list so 'n'.join() doesn't crash.
31	<code>conversation = [SystemMessage(content=system_content)]</code>	Starts the conversation with the system prompt containing the advice and plan context.

Line	Code	Explanation
32	<code>for message in state["messages"][:-1]:</code>	Loops through the full conversation history (except the current message) to give the LLM context about what's already been discussed. This prevents: 'How are you feeling?' being asked in turn 1, turn 2, and turn 3.
39	<code>conversation.append(HumanMessage(content=state["messages"][-1]["content"]))</code>	Adds the current user message as the final HumanMessage. The LLM generates the follow-up question in response to this complete conversation context.
41	<code>response = llm.invoke(conversation)</code>	Sends the full conversation to the LLM. Returns a single follow-up question.
42	<code>state["follow_up_question"] = response.content</code>	Stores the question. The frontend displays it separately from the main advice — typically below the action plan, styled as a conversation prompt.
43	<code>state["turn_count"] += 1</code>	Increments the turn counter. Tracks how many full conversation rounds have completed. Could be used in future to limit turns or change behavior (e.g. suggest professional help after 5 turns).
44	<code>state["should_loop"] = False</code>	Sets the loop flag to False — this turn is complete. The graph proceeds to END and the full response (advice + action plan + follow-up) is sent to the frontend.
45	<code>return state</code>	Returns state. Graph routes to END (fixed edge). The pipeline is complete — <code>api/main.py</code> sends the <code>'__end__'</code> SSE event to the frontend.

13. rag/ingest.py — PDF Ingestion Pipeline

backend/rag/ingest.py

A one-time script that builds the ChromaDB vector database. Reads all 6 PDF books → splits into 800-character chunks with 120-char overlap → filters out junk (ads, index pages, short headers) → converts each chunk into a 384-dimensional vector using all-MiniLM-L6-v2 → stores in ChromaDB with cosine similarity. You run this once; the database persists on disk for all future server runs.

Line	Code	Explanation
1	<code>import os</code>	For file path operations — constructing paths to the PDF files and the .env file.
2	<code>import sys</code>	For <code>sys.path.insert()</code> which modifies Python's module search path so imports work correctly when running this script directly.
3	<code>import re</code>	For regular expressions used in junk detection patterns (<code>_JUNK_PATTERNS</code> and <code>_INDEX_ENTRY</code>).
6	<code>sys.path.insert(0, ...)</code>	Adds the backend/ directory to Python's import path. When you run 'python rag/ingest.py' directly, Python's working directory is different from when the server runs, so normal imports like 'from rag.retriever' would fail. This line fixes the import path so the script can be run standalone.
8	<code>from dotenv import load_dotenv</code>	Loads the .env file for any API keys. While ingest.py doesn't call the Groq API, loading the environment is good practice in case future code needs it.
11	<code>from langchain_community.document_loaders import PyPDFLoader</code>	Imports the PDF reader from LangChain's community package. PyPDFLoader reads a PDF file page by page and returns a list of Document objects, each with .page_content (the extracted text) and .metadata (page number, source file path).
12	<code>from langchain_text_splitters import RecursiveCharacterTextSplitter</code>	Imports the text splitter. 'Recursive' means it tries to split at the most natural boundary: first paragraphs (<code>\n\n</code>), then sentences (<code>.</code>), then words (<code> </code>), then characters. This produces chunks that break at sentence boundaries rather than mid-word.
13	<code>from langchain_huggingface import HuggingFaceEmbeddings</code>	Imports the embedding model wrapper. This loads a HuggingFace transformer model locally and converts text strings into numerical vectors (lists of 384 floating-point numbers).
14	<code>import chromadb</code>	Imports ChromaDB directly (not through LangChain) because we need <code>create_collection()</code> with specific settings like cosine similarity. LangChain's Chroma wrapper is used in rag/retriever.py for querying, but for initial database creation we need more control.
20	<code>_JUNK_PATTERNS = [...]</code>	List of regex patterns that detect promotional/ad text embedded in PDF books. Some book PDFs include ads for companion workbooks, coupon codes, or website links between chapters. Examples: 'coupon\\s+code' matches 'coupon code', 'save\\s+an?\\s+additional' matches 'save an additional 20%'. If any pattern matches a chunk, it's discarded.
30	<code>_INDEX_ENTRY = re.compile(r'\b[\w\s]+, \s+\d+[--]\d+')</code>	Compiled regex that matches book index entries like 'motivation, 42-45' or 'self-esteem, 100-103'. Compiled once at module level for performance (not recompiled on every call). If a chunk has more than 4 matches, it's an index page — pure page references, no useful content for advice.
33	<code>def _is_content(text):</code>	Filter function that decides if a chunk is real book content (return True) or junk to discard (return False). Three sequential checks, each catching a different type of junk.
36	<code>if len(text.strip()) < 120: return False</code>	Filter 1: Length check. Chunks shorter than 120 characters are usually headers ('Chapter 5'), footers ('Page 87'), or title pages. They contain too little content to be useful for advice — the psychologist can't cite a 10-word header. 120 chars ≈ 1-2 sentences minimum.

Line	Code	Explanation
39	for pattern in _JUNK_PATTERNS: if re.search(...):	Filter 2: Junk pattern check. Scans the lowercased text for any promotional pattern. re.search() finds the pattern anywhere in the text (not just at the start). If any pattern matches, the chunk is rejected.
42	if len(_INDEX_ENTRY.findall(text)) > 4: return False	Filter 3: Index page check. Counts how many index-style entries appear. A real content chunk might mention one page reference in passing, but a chunk with 5+ entries like 'anxiety, 23-25\nattachment, 45-52\navoidance, 67-71' is clearly an index page.
44	return True	Passed all three filters — this is genuine book content suitable for the knowledge base.
47	BOOKS = {...}	Dictionary mapping 6 book display names to their PDF file paths. The display name (e.g. 'Feeling Good') becomes the 'source' metadata stored with each chunk in ChromaDB, which is how the retriever and frontend know which book a chunk came from. Adding a new book to the system means adding one entry here and re-running ingest().
55	def ingest():	The main function that orchestrates the entire ingestion pipeline: load model → read PDFs → split → filter → embed → store.
63	embeddings = HuggingFaceEmbeddings(...)	Loads the all-MiniLM-L6-v2 embedding model locally on CPU. This model converts text into 384-dimensional vectors. 'normalize_embeddings=True' scales all vectors to unit length so cosine similarity produces values between 0 and 1. Critical: this MUST be the same model used in rag/retriever.py — if you embed with model A and search with model B, the vector spaces don't match and search returns garbage.
71	splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=120)	Creates the text splitter. chunk_size=800: each chunk is ~800 characters (~150-200 words) — large enough to contain a complete concept or technique, small enough to be focused. chunk_overlap=120: the last 120 chars of chunk N are also the first 120 chars of chunk N+1. This overlap prevents sentences from being cut in half at boundaries — if an important sentence straddles two chunks, both chunks contain it.
72	all_chunks = []	Accumulator list for all clean chunks across all 6 books. After the loop, this contains ~1500 chunks ready for embedding.
75	for book_name, pdf_path in BOOKS.items():	Loops through all 6 books. Each iteration: load PDF → split → tag → filter → accumulate.
77	loader = PyPDFLoader(pdf_path)	Creates a PDF reader for this specific book file.
78	pages = loader.load()	Reads the entire PDF into memory. Returns a list of Document objects, one per page. A 300-page book produces 300 Document objects, each with the full page text and page number metadata.
79	chunks = splitter.split_documents(pages)	Splits all pages into ~800-character chunks. A 300-page book might produce ~1500 chunks. The splitter respects paragraph and sentence boundaries where possible.
81	chunk.metadata["source"] = book_name	Tags every chunk with the book's display name. This is how the entire system (retriever filtering, evaluator logging, psychologist citations, frontend sources display) knows which book a chunk came from. The original metadata only has the file path and page number — this adds the human-readable name.
82	clean_chunks = [c for c in chunks if _is_content(c.page_content)]	Applies the _is_content filter to every chunk. Typically removes 5-15% of chunks (headers, footers, ads, index pages). The remaining chunks are real, substantive book content.
91	client = chromadb.PersistentClient(path="./chroma_db")	Creates a ChromaDB client that stores data on disk at ./chroma_db/. 'Persistent' means the data survives server restarts — you ingest once and the database remains. Without persistent storage, you'd need to re-ingest all 6 books every time the server starts.
92	collection = client.create_collection(...)	Creates a new collection (like a database table) named 'langchain' with cosine similarity. hnsw:space=cosine tells ChromaDB to use the HNSW (Hierarchical Navigable Small World) algorithm with cosine distance — an efficient approximate nearest-neighbor search that scales to millions of vectors.

Line	Code	Explanation
101	<code>vectors = embeddings.embed_documents(texts)</code>	Converts ALL chunk texts into vectors in one batch. Each text string becomes a list of 384 floating-point numbers. This is the most time-consuming step — embedding ~1500 chunks takes 30-60 seconds on CPU. Batching is much faster than embedding one at a time.
104	<code>batch_size = 500</code>	Insert chunks 500 at a time to avoid memory issues. ChromaDB processes each batch atomically — if you tried to insert all ~1500 at once, it might run out of memory on smaller machines.
105	<code>for start_index in range(0, len(texts), batch_size):</code>	Loops in steps of 500: 0-499, 500-999, 1000-1499, etc.
106	<code>collection.add(ids=..., embeddings=..., documents=..., metadatas=...)</code>	Inserts one batch into ChromaDB. Each chunk is stored with 4 pieces of data: a unique string ID, its vector embedding (for similarity search), the original text (returned in search results), and metadata (book name, page number). All four arrays must be in the same order.
117	<code>if __name__ == "__main__":</code>	Python guard — only runs when you execute 'python ingest.py' directly, not when imported. This lets other files import <code>_is_content()</code> or <code>BOOKS</code> without triggering the full ingestion process.
118	<code>ingest()</code>	Calls the main function. The entire ingestion takes 1-3 minutes depending on your machine speed.

14. rag/retriever.py — Vector Store Initialization

backend/rag/retriever.py

Initializes the embedding model and ChromaDB connection ONCE at import time (when the server starts). Wraps ChromaDB into a LangChain retriever interface so it can be called with `.invoke(query)` → top 8 chunks. This is a short but critical bridge file — it connects the ChromaDB database (created by `ingest.py`) to the agent pipeline. Must use the SAME embedding model as `ingest.py` or search returns garbage.

Line	Code	Explanation
1	<code>from langchain_chroma import Chroma</code>	Imports LangChain's Chroma wrapper — different from the raw 'import chromadb' used in <code>ingest.py</code> . The LangChain wrapper provides <code>.as_retriever()</code> which creates a standard LangChain retriever interface, and <code>.similarity_search_with_score()</code> used by the <code>/db/search</code> API endpoint.
2	<code>from langchain_huggingface import HuggingFaceEmbeddings</code>	Imports the same embedding model wrapper used in <code>ingest.py</code> . The search-time model MUST match the ingestion-time model — embeddings from different models live in different mathematical spaces.
5	<code>_embeddings = HuggingFaceEmbeddings(...)</code>	Loads the all-MiniLM-L6-v2 model at module level (import time). This takes 2-3 seconds and ~80MB of memory. Loading once and reusing for all requests is critical — if this were inside a function called per-request, every search would have a 2-3 second delay. The underscore prefix (<code>_</code>) signals this is a module-internal variable, not meant to be imported directly by other modules.
10	<code>_vectorstore = Chroma(persist_directory="./chroma_db", embedding_function=_embeddings)</code>	Connects to the existing ChromaDB database on disk (created by <code>ingest.py</code>). <code>persist_directory</code> points to the same <code>./chroma_db/</code> folder. <code>embedding_function</code> tells the wrapper which model to use for converting search queries into vectors. If the <code>chroma_db</code> folder doesn't exist (you haven't run <code>ingest.py</code>), this will create an empty database and searches will return no results.
14	<code>_retriever = _vectorstore.as_retriever(search_type="similarity", search_kwargs={"k": 8})</code>	Wraps the vectorstore into a LangChain retriever interface . When you call <code>_retriever.invoke('feeling dismissed')</code> , it: (1) converts 'feeling dismissed' into a 384-dim vector using <code>_embeddings</code> , (2) asks ChromaDB to find the 8 vectors closest to it by cosine distance, (3) returns the 8 corresponding Document objects with <code>.page_content</code> and <code>.metadata</code> . The <code>k=8</code> means we always retrieve 8 chunks — the evaluator then judges if they're good enough.
20	<code>def build_retriever():</code>	Simple getter function. Returns the pre-initialized retriever. Called by <code>agent/nodes/retriever.py</code> at import time to get the retriever object without needing to know the initialization details.
21	<code>return _retriever</code>	Returns the module-level retriever. Since everything was created at import time, this is essentially free — no computation, just returning a reference to an existing object.

15. api/main.py — FastAPI Server & Endpoints

backend/api/main.py

The FastAPI application — the HTTP layer that connects the frontend to the agent pipeline. Defines the /chat endpoint (streams pipeline results via SSE as each node completes), three /db/ endpoints for debugging the vector database, and two utility endpoints. Handles CORS (so the React frontend on a different port can call the API), state initialization, and JSON serialization.

Line	Code	Explanation
1	<code>from fastapi import FastAPI, Query</code>	FastAPI is the web framework — it handles HTTP routing, request parsing, response formatting, and auto-generates API documentation. Query is used to define and validate URL query parameters (like <code>?query=...&top_k=8</code>) on the /db/ endpoints.
2	<code>from fastapi.middleware.cors import CORSMiddleware</code>	Imports CORS (Cross-Origin Resource Sharing) middleware. Browsers block requests from one origin (<code>http://localhost:3000</code> , the React app) to another origin (<code>http://localhost:8000</code> , the API) by default — this is a security feature. CORS middleware tells the browser 'it's OK, these origins are allowed to call me.'
3	<code>from fastapi.responses import StreamingResponse</code>	Imports <code>StreamingResponse</code> — a FastAPI class that sends data to the client in chunks as they become available, instead of building the entire response in memory first. Essential for SSE streaming where each node's result is sent as soon as it completes.
4	<code>from pydantic import BaseModel</code>	Imports Pydantic's <code>BaseModel</code> for defining request body schemas. FastAPI uses Pydantic models to automatically parse JSON request bodies, validate field types, and generate API documentation.
5	<code>from typing import List</code>	Imports <code>List</code> for type annotations in the Pydantic models.
6	<code>from agent.graph import build_graph</code>	Imports the function that builds and compiles the LangGraph pipeline. This import triggers the entire chain: <code>graph.py</code> imports all node modules → each node imports <code>llm_factory</code> → <code>llm_factory</code> creates LLM objects. So this single import line boots up the entire agent system.
7	<code>from agent.state import AgentState</code>	Imports the state <code>TypedDict</code> for creating the initial state in the /chat endpoint.
8	<code>from rag.retriever import _vectorstore</code>	Imports the raw ChromaDB vectorstore object (not the retriever wrapper) for the /db/ debugging endpoints. These endpoints need <code>.get()</code> (to list all data) and <code>.similarity_search_with_score()</code> (to test queries with scores) — methods not available through the retriever interface.
9	<code>import json</code>	Imports Python's JSON library for <code>json.dumps()</code> — used to convert Python dicts into JSON strings for SSE events sent to the frontend.
11	<code>app = FastAPI(title="PsycheNavigator API")</code>	Creates the FastAPI application object — the central object that uvicorn runs. The title appears in the auto-generated API documentation at /docs.
12	<code>graph = build_graph()</code>	Compiles the LangGraph pipeline ONCE at server startup. This returned object is reused for every request — it's a template that gets executed with fresh <code>initial_state</code> each time. Building the graph is fast (milliseconds) but loading all the LLM objects and embedding models during the import chain takes several seconds.
16	<code>app.add_middleware(CORSMiddleware, ...)</code>	Enables CORS for the React frontend. allow_origins lists the URLs that can call the API: <code>localhost:3000</code> (Create React App default), <code>localhost:3001</code> (CRA alternate), <code>localhost:5173</code> (Vite default). allow_methods=["*"] allows all HTTP methods. allow_headers=["*"] allows all request headers. Without this middleware, every frontend <code>fetch()</code> call would fail with a CORS error in the browser console.

Line	Code	Explanation
27	<code>class Message(BaseModel):</code>	Defines one chat message for the request body. Pydantic validates that both 'role' (str) and 'content' (str) are present and are strings. If the frontend sends a message without a role field, FastAPI returns a 422 Validation Error automatically.
31	<code>class ChatRequest(BaseModel):</code>	Defines the full request body: a list of Message objects. The frontend sends the ENTIRE conversation history on every request (not just the new message) — this is the standard pattern for stateless APIs where the server doesn't store session data.
34	<code>MAX_HISTORY = 10</code>	Limits conversation history to the last 10 messages. In a long conversation (30+ messages), sending all of them would waste tokens and might exceed the LLM's context window. 10 messages (5 user + 5 assistant) provides enough context for multi-turn conversations while keeping token usage manageable.
38	<code>@app.post("/chat")</code>	Registers this function as the handler for POST requests to /chat. This is the main endpoint — every user message from the frontend comes through here.
39	<code>async def chat(request: ChatRequest):</code>	The handler function. async because it uses SSE streaming — the server sends data incrementally while keeping the connection open. FastAPI automatically parses the JSON request body into a ChatRequest object.
40	<code>recent_messages = request.messages[-MAX_HISTORY:]</code>	Slices to keep only the last 10 messages. If the conversation has 25 messages, the first 15 are dropped. Python's [-10:] slice gracefully handles lists shorter than 10 (returns the full list).
41	<code>initial_state: AgentState = {...}</code>	Creates a fresh state for this request. Every field is zeroed out: empty chunks, 0 attempts, no response, not off-topic. This is the 'construction' step — state.py defined the blueprint (types), this code fills in the initial values. A new state is created for every request — requests don't share state.
42	<code>"messages": [message.model_dump() for ...]</code>	Converts Pydantic Message objects into plain Python dicts using <code>model_dump()</code> . This is necessary because the node code expects regular dicts (<code>msg['role']</code>), not Pydantic objects (<code>msg.role</code>). <code>model_dump()</code> turns <code>Message(role='user', content='hello')</code> into <code>{'role': 'user', 'content': 'hello'}</code> .
43	<code>"current_query": recent_messages[-1].content</code>	Sets the initial search query to the latest message's text. This is what the retriever will search ChromaDB with on the first attempt. If the evaluator rejects chunks, the <code>query_builder</code> will overwrite this with a rephrased version.
58	<code>async def stream():</code>	An async generator function . Each 'yield' sends one SSE event to the frontend immediately, without waiting for the rest of the pipeline. This is what enables real-time progress — the frontend shows 'Guard ✓' as soon as the guard completes, before the retriever even starts.
60	<code>async for graph_output in graph.astream(initial_state):</code>	Runs the LangGraph pipeline with the initial state. graph.astream() is an async iterator — it yields one result each time a node completes. The 'async for' loop automatically waits for each node to finish before processing its output. This is the 'wait' mechanism — there's no explicit sleep or poll, the async iteration handles it.
61	<code>for node_name, state_update in graph_output.items():</code>	Each <code>graph_output</code> is a dict like <code>{'guard': {'is_off_topic': False, 'current_query': '...'}}</code> . We extract the node name ('guard') and the state fields it updated. A single node might update multiple fields.
63	<code>serializable_fields = {...}</code>	Filters the state update to keep only JSON-safe types: str, int, float, bool, list, dict, None. Drops any LangChain-specific objects (AIMessage, Document, Runnable) that can't be converted to JSON. Without this filter, <code>json.dumps()</code> would crash on non-serializable types and the entire SSE stream would break.
67	<code>yield f"data: {json.dumps({...})}\n\n"</code>	Sends one SSE event to the frontend. The format <code>'data: {JSON}\n\n'</code> is the standard SSE wire protocol — 'data:' prefix, JSON payload, double newline to mark end of event. The frontend's EventSource API parses this automatically. Each event tells the frontend which node just completed and what it produced.

Line	Code	Explanation
68	<code>yield "data: {\\"node\\": \\"__end__\\"}\n\n"</code>	Sends the final event with the special node name ' <code>__end__</code> '. The frontend listens for this to know the pipeline is done — it stops the loading spinner and enables the input field for the next message.
70	<code>return StreamingResponse(stream(), media_type="text/event-stream")</code>	Returns the generator as a streaming HTTP response. <code>media_type='text/event-stream'</code> tells the browser this is an SSE stream — the browser keeps the connection open and processes events as they arrive instead of waiting for the entire response. Without the correct media type, the browser would buffer everything and only display results after the pipeline finishes.
76	<code>@app.get("/db/stats")</code>	GET /db/stats — debugging endpoint that shows how many chunks each book has in ChromaDB. Calls <code>_vectorstore.get()</code> to pull ALL data, counts chunks per book, sorts by count (largest first), returns total + per-book breakdown. Used to verify that ingestion worked correctly.
102	<code>@app.get("/db/search")</code>	GET /db/search?query=...&top_k=8 — debugging endpoint that tests a query against ChromaDB directly (bypassing the entire agent pipeline). Returns the top-k chunks with their cosine distance scores (0 = perfect match, 1 = completely unrelated). Invaluable for debugging retrieval: 'Why did the evaluator reject these chunks? Let me see what ChromaDB actually returns and how similar the scores are.'
120	<code>@app.get("/db/browse")</code>	GET /db/browse?book=Feeling Good&offset=0&limit=20 — debugging endpoint that lets you page through all chunks for a specific book. Filters all chunks by book name, then returns a paginated slice. Useful for inspecting actual chunk content: 'What does chunk #145 of Nonviolent Communication look like?'
143	<code>@app.get("/")</code>	GET / — Landing page. Returns the API name, status, and available endpoints as JSON. If someone visits <code>http://localhost:8000/</code> in a browser, they see useful info instead of a 404 error. Standard practice for REST APIs.
152	<code>@app.get("/health")</code>	GET /health — Returns <code>{'status': 'ok'}</code> if the server is alive and responsive. Used by deployment platforms (Docker, Kubernetes, AWS ECS) that periodically ping this endpoint and restart the server if it stops responding. Also useful for manual checks: <code>'curl localhost:8000/health'</code> to verify the server is running.

End of Document

Every line of every backend file explained in detail. 15 files, ~780 lines of Python.

Each explanation covers: what the line does, why it's needed, what would break without it, and how it connects to other files in the system.